

Проект №2275 ИСП РАН
Тестовый стенд для исследования тестовых наборов
для микропроцессоров с архитектурой RISC-V

Цель проекта

Создание автоматизированного тестового стенда для сравнительного анализа существующих тестовых наборов RISC-V. Система **не разрабатывает** новые тесты и симуляторы — она запускает **готовые тестовые наборы на готовых симуляторах** (Spike, QEMU), собирает метрики покрытия кода, делает *cross/cov* и формирует сравнительные отчёты. Вся работа скриптов автоматизирована (Bash/Python).

- ✓ Готовые тестовые наборы
- ✓ Готовые симуляторы
- ✓ gcov/lcov для покрытия
- ✓ Автоматизация (скрипты)

1 Что мы сравниваем?

Тестовые наборы — существующие коллекции тестов для RISC-V. Каждый набор запускается на **одном и том же симуляторе** (Spike, скомпилированном с флагами — coverage), после чего gcov/lcov собирает данные о покрытии кода симулятора.

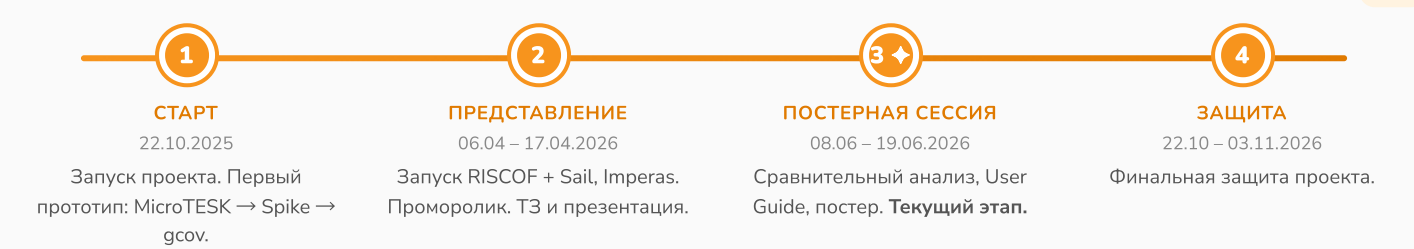
MicroTESK АЛГОРИТМИЧЕСКИЕ ТЕСТЫ 30 ELF-программ (сортировка, деление, работа с регистрами), Генератор ИСП РАН.	RISCOF АРХИТЕКТУРНЫЕ ТЕСТЫ ISA 63 ассемблерных файла. Официальный фреймворк RISC- V International.	Imperas СИГНАТУРНЫЕ ТЕСТЫ 48 ассемблерных файлов. Сравнение сигнатур с коммерческим эталоном riscvOVPsim.
RV64I	RV32I + RV64I + M	RV32I (откр.)

2 По каким критериям сравниваем?

Согласно ТЗ (раздел 5.2), основная метрика — **покрытие исходного кода** симуляторов (Code Coverage). Система рассчитывает для каждого тестового набора следующие показатели:

- 1 Line Coverage (покрытие строк)** Процент строк исходного кода симулятора Spike, выполненных при прогоне тестового набора. **Источники:** [gcc/llvm](#).
- 2 Function Coverage (покрытие функций)** Процент функций симулятора, которые были вызваны хотя бы один раз в процессе исполнения тестов.
- 3 Относительная эффективность** Прямое сопоставление процента покрытия: «Набор N1» vs «Набор N2» на одном и том же симуляторе (Spike).
- 4 Delta Coverage (инкрементальный анализ)** Анализ кода симулятора, покрываемые «Набором N2», но не затронутые «Набором N1». Выявляет уникальную ценность каждого набора.

Циклы реализации проекта



Архитектура тестового стенда

Два Docker-контейнера. Симулятор Spike скомпилирован с флагами `---coverage` для сбора gcov-данных. Все инструменты предустановлены.

```
graph LR; A["1. Запуск тестов  
Spike  
gcov  
Sail C"] --> B["2. Сбор покрытия  
gcov собирает  
данные  
lcov -> info"]; B --> C["3. Анализ  
LinFunction Coverage  
Delta Coverage  
Сравнительный отчет"]; subgraph Tools; D["Docker"]; E["Bash/Python"]; F["lcov"]; end; subgraph CI_Environment ["CI-среда"]; G["Spike + gcov"]; H["RISCOP"]; I["MicroTESK"]; J["Imperas"]; end;
```

1. Запуск тестов
Скрипты Bash/Python прогоняют все тесты

2. Сбор покрытия
gcov собирает данные
lcov → info

3. Анализ
LinFunction Coverage
Delta Coverage
Сравнительный отчет

CI-среда: Spike + gcov, RISCOP, MicroTESK, Imperas

Инструменты: Docker, Bash/Python, lcov

Результаты: покрытие кода Spike (gcov/lcov)

ПОКРЫТИЕ КОДА SPIKE (LINE / FUNCTION COVERAGE) — ЕДИНАЯ МЕТОДИКА

Тестовый набор	ISA	ELF	Line Cov.	Func Cov.
MicroTESK	RV64I	30	16.2%	12.2%
RISCOF	RV64I	50	15.5%	11.8%
Imperas	RV32I	48	15.7%	12.3%

16.2%
Line Coverage

MicroTESK

15.7%
Line Coverage

Imperas

15.5%
Line Coverage

RISCOF RV64I

Delta Coverage: все три набора дают схожий общий процент, но покрывают **разные участки** кода Spike. MicroTESK (алгоритмические тесты) и RISCOF/Imperas (поинструкционные) дополняют друг друга.

Методология сбора и сравнения

КАК ЭТО РАБОТАЕТ

1. **Инструментация симулятора.** Spike компилируется с флагами `-coverage (GCC)`. При каждом запуске теста генерируются файлы `.gcda` / `.gcno` с данными о выполненных строках кода.
2. **Автоматический прогон.** Скрипты Bash/Python последовательно запускают все тесты набора на инструментированном Spike. Перед каждым новым набором счётчики `gcov` очищаются.
3. **Сбор метрик.** `lcov` агрегирует `.gcda`-файлы и генерирует отчёт: Line Coverage (%), Function Coverage (%), а также детализацию по каждому файлу исходного кода Spike.
4. **Сравнительный анализ.** Прямое сопоставление процентов покрытия у инкрементального Delta-анализа: какие строки покрыты Набором №2, но не затронуты Набором №1.

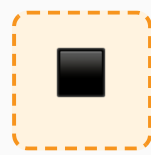
Текущий статус и дальнейшие шаги

ЧТО УЖЕ СДЕЛАНО

- Развёрнут тестовый стенд в двух Docker-контейнерах. Скрипт скомпилирован с —coverage. Все три тестовых набора запускаются на едином инструментированном Spike.
- Собраны данные Line/Function Coverage (gcov/lcov) для всех трёх наборов по одной методике: MicroTST 16.2%/12.2%, Imperas 5.7%/12.3%, RISCOS RV64I 15.5%/11.8%.
- Написаны скрипты автоматизации (Bash) — полный цикл: очистка счётчиков → прогон тестов → lcov → HTML-отчёт.

ДАЛЬНЕЙШИЕ ШАГИ

- Вычисление Delta Coverage — уникального покрытия каждого набора относительно других. Определение участков кода Spike, покрываемых только одним из наборов.



git.miem.hse.ru/2275/risc-v